This output is designed to help you find at a glance what went wrong and how. As we'll see next, RSpec can provide further cues through syntax highlighting.

Syntax Highlighting

We've seen how RSpec's color highlighting makes it *much* easier to scan the output for passing and failing specs. We can take it a step further by installing a code highlighter called CodeRay:[1]

```
$ gem install coderay -v 1.1.1
Successfully installed coderay-1.1.1
1 gem installed
```

When this gem is installed, the Ruby snippets in your specs' output will be color-coded just like they'd be in your text editor. For example:

```
$ rspec -fd

A cup of coffee
  costs $1
  with milk
    costs $1.25 (FAILED - 1)

Failures:

  1) A cup of coffee with milk costs $1.25
     Failure/Error: expect(coffee.price).to eq(1.25)

       expected: 1.25
            got: 1.0

       (compared using ==)
     # ./spec/coffee_spec.rb:26:in `block (3 levels) in <top (required)>'

Finished in 0.0102 seconds (files took 0.09104 seconds to load)
2 examples, 1 failure

Failed examples:

rspec ./spec/coffee_spec.rb:25 # A cup of coffee with milk costs $1.25
```

Now, the line `expect(coffee.price).to eq(1.25)` has Ruby syntax highlighting. Normal method calls like `coffee` and `price` aren't shaded, but other elements are. In particular, both the key RSpec `expect` method and the number `1.25` are highlighted in color. This syntax highlighting is even more helpful for complex Ruby expressions.

RSpec will automatically use CodeRay if it's available. For Bundler-based projects, drop it into your `Gemfile` and rerun `bundle install`. For non-Bundler projects, install it via `gem install` as we've done here.

1. https://github.com/rubychan/coderay